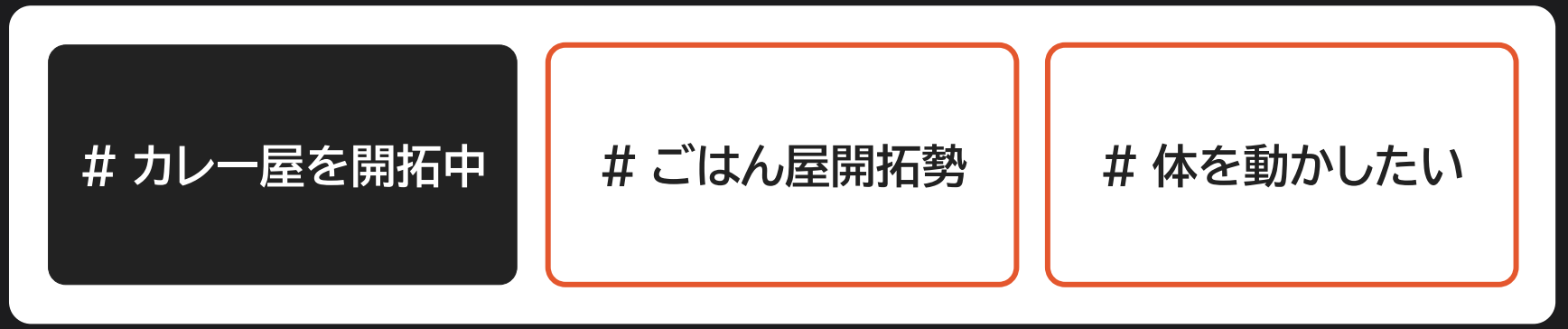
おいとま活用中

名札に「話題」を足して、肩書き以外の話から始める

学内 AI おいとまで、交流会の自己紹介を話題シールに



個性タグ 1
＋ 共有タグ 2

（説明用の架空例）

課題

話題づくりに AI を使いたい、でも回答は個人情報



名札にあるのは名前と所属だけ

01 話のきっかけが、名札からは見えない

02 数十～数百人の自己紹介から共通点を探すのは、手作業では難しい

03 自己紹介は個人情報。学外の AI には渡しにくい

つくるもの

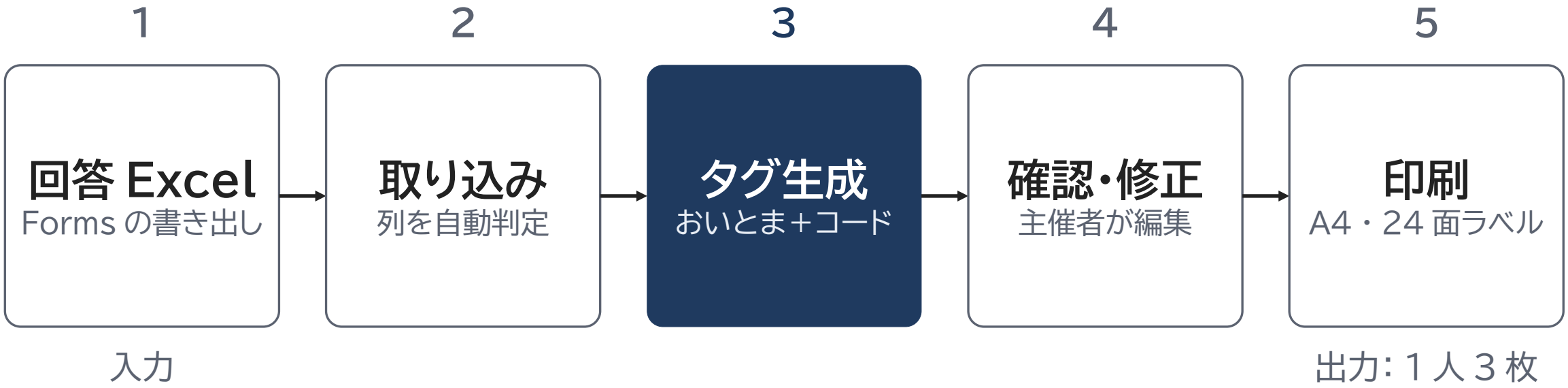
回答から、会話の入口になる 3 枚のシールを作る



表現が違っても、**意味の近い話題をまとめる**

処理の流れ

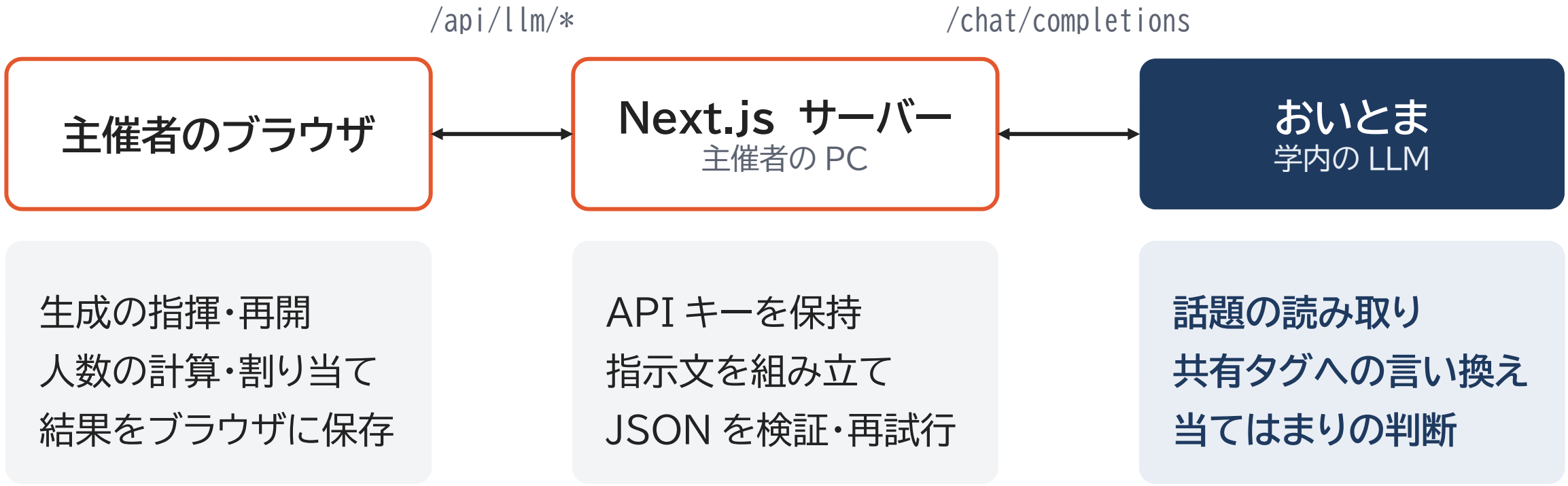
Excel を読み込めば、印刷まで主催者の画面で完結



生成のあと、**主催者が確認・修正してから印刷する**

おいとまの役割

おいとまは「言葉の処理」だけを担当する



意味の判断は **AI** 、数の計算と保存は**アプリ**

呼び出すタイミング

おいとまを呼ぶのは、生成・確認・当日追加の３場面

事前　生成画面

タグをまとめて作る

↓

話題・候補・順位・まとめ

全員分のタグができる

確認　確認・修正画面

おいとまに別案を出してもらう

↓

個性タグの別案を３つ

候補に追加する

当日　取り込み画面

追加してタグを作る

↓

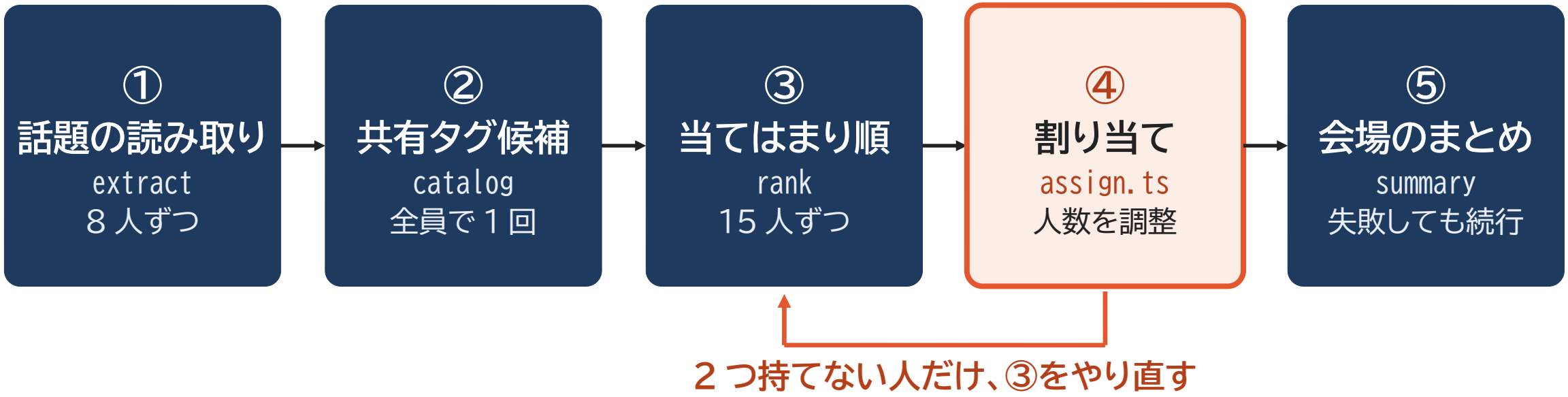
その１人だけ抽出・順位

他の人のタグは変えない

保存済みの結果を開く・印刷するときは呼ばない

主要な処理

生成は 5 段階：言葉はおいとま、割り当てはコード



技術的な工夫

割り当てはAIに任せず、コードで人数を守る

	比較案（未実装） **AIに全部任せる** AIが全員分の割り当てまで出力	**採用** **AIとコードで分担** AIは候補と順位、割り当てはコード
出力	全員分を一度に	**数人分ずつ短く**
人数の下限	守れたか分からない	**コードが守る**
失敗時	全体をやり直し	**その分だけ再試行**

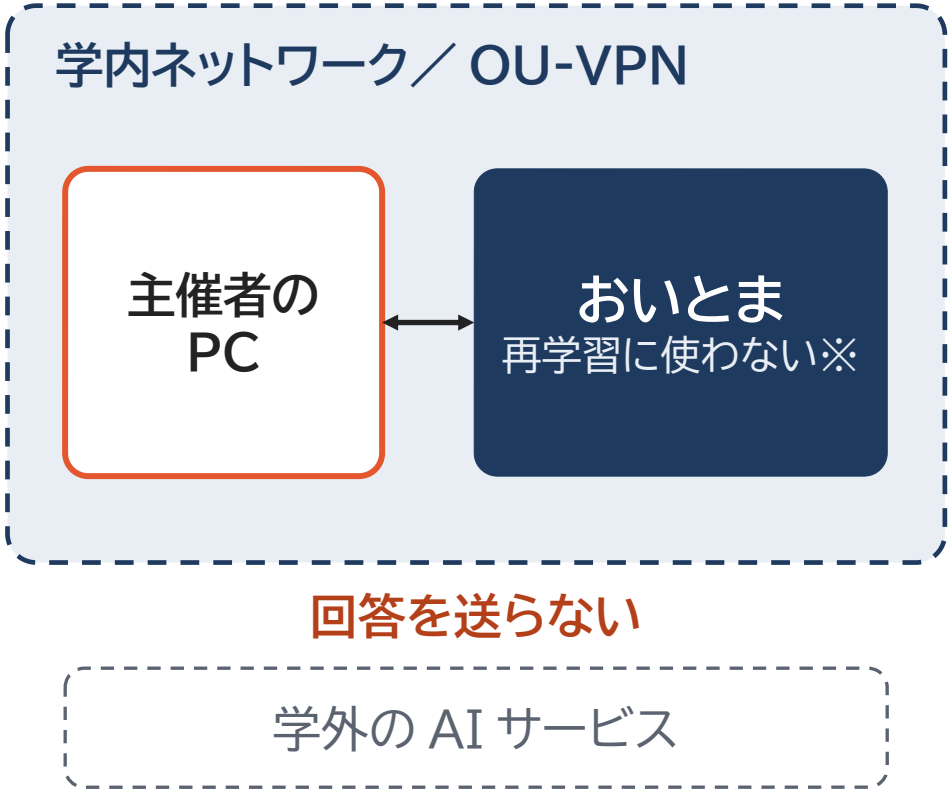
数の制約は、**テストで確かめられるコード**で守る

おいとまを使う意義

学内 AI だから、個人情報をセキュアに AI 処理できる

参加者の自己紹介を、
学外の AI 事業者に渡さず AI 処理

- 名前は送らず、ID で処理
- 同意なしの自己紹介は送らない
- サーバーに保存しない



※ 仕様書の記載（根拠はおいとまマニュアル）。学外の端末から使う場合は付録 A3

結果

取り込みから印刷まで、動作を確認した

98/98

自動テスト成功

17 ファイル・10/15 実行

18/18

受け入れ確認

基本 9 ＋追加 9・mock

120 枚

印刷 PDF を検査

40 人分・A4 5 ページ

本物のおいとま(live)の出力を見て、指示文を改善

未計測：会場での会話の効果、live の所要時間

学内 AI で、名札に「話題」を足す

1. 言葉はおいとま、人数はコードで分担
2. Excel から印刷まで、1 つの画面で完結
3. 個人情報を、学内 AI でセキュアに処理

おいとま活用中

下限に届かないタグを 1 つずつ外し、割り当て直す

```
for (let iter = 0; iter < candidateIds.length; iter += 1) {
  pickTop2();
  resolveOverflow();
  const counts = countHolders();
  active = active.filter((id) => (counts.get(id) ?? 0) > 0);
  const recount = countHolders();
  const below = active.filter(
    (id) => (recount.get(id) ?? 0) < minSize);
  if (below.length === 0) break;
  let victim = below[0]; // … 最少人数のタグを選ぶ
  active = active.filter((id) => id !== victim);
}
```

src/lib/assign.ts 233–251 行(「…」は省略、改行位置のみ変更)

01　各人に、順位の上位 2 つを持たせる

02　下限未満のタグから最少の 1 つを外す

03　下限未満がなくなるまで繰り返す

崩れた JSON は、理由を添えて最大 3 回まで再依頼

```
try {
  const text = extractJsonText(contentStr);
  const parsed: unknown = JSON.parse(text);
  const r = opts.schema.safeParse(parsed);
  if (!r.success) { /* … */ throw new Error(…); }
  // …validate で追加検査、成功なら return
} catch (e) {
  const reason = e instanceof Error ? e.message : String(e);
  if (attempts >= 3) { /* … */ throw new LlmError(…); }
  messages = [...messages,
    { role: 'assistant', content: contentStr.slice(0, 2000) },
    { role: 'user', content:
      `前の出力は条件を満たしていません：${reason}。…` }];
}
```

src/lib/llm/client.ts 280–311 行(「…」は省略、改行位置のみ変更)

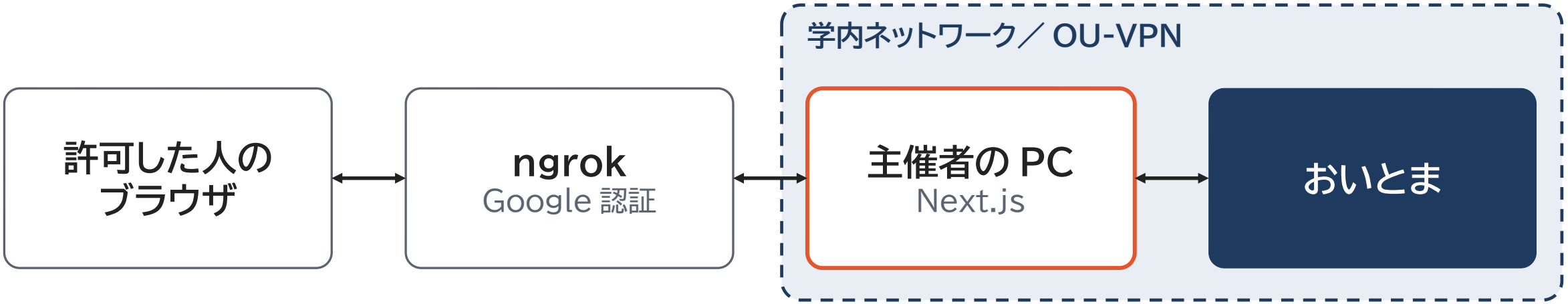
01 思考の欄やコード記号を除いて JSON を取り出す

02 zod で形式を検証する

03 失敗理由を会話に足して再依頼する

付録 A3　学外からの利用

学外からは、ngrok の Google 認証を通して使う



01　AI の処理は学内。ただし通信は ngrok を経由する

02　手順書で、通信内容を残さない設定にする

03　認証の動作確認は、資料上まだ記録がない